1. **Aim of the practical:**
   Write a program based on Artificial neural (using Tensor flow). Also specify the data set used with number of input and output variables.

2. **Objective:**
   - To understand how artificial neural networks (ANNs) can classify non-linear data distributions.
   - To apply TensorFlow and Keras libraries for building and training a simple feedforward neural network.
   - To visualize and compare the decision boundaries formed by the model for different types of datasets — *moons*, *circles*, and *blobs*.

3. **Algorithm/Flowchart:**

A. circles:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_moons, make_circles, make_blobs
from sklearn.model_selection import train_test_split
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```

```
data, target = make_circles(n_samples=1000, noise=0.03)
```

```
plt.figure(figsize=(5,5))
plt.scatter(data[:,0], data[:,1], c=target, cmap='viridis')
plt.title("Dataset Visualization")
plt.show()
```

```
x_train, x_test, y_train, y_test = train_test_split(data, target, test_size=0.25, random_state=42)
```

```
model = Sequential([
    Dense(8, activation='relu', input_shape=(2,)),
    Dense(4, activation='relu'),
    Dense(1, activation='sigmoid')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.1),
              loss=tf.keras.losses.BinaryCrossentropy(),
              metrics=['accuracy'])
```

```
history = model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
```

```
Epoch 1/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5279 - loss: 0.6925 - val_accuracy: 0.4560 - val_loss: 0.6945
Epoch 2/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5372 - loss: 0.6921 - val_accuracy: 0.4560 - val_loss: 0.6940
Epoch 3/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5187 - loss: 0.6929 - val_accuracy: 0.4560 - val_loss: 0.6958
Epoch 4/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5285 - loss: 0.6921 - val_accuracy: 0.4560 - val_loss: 0.6956
Epoch 5/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5178 - loss: 0.6929 - val_accuracy: 0.4560 - val_loss: 0.6958
```

```
loss, acc = model.evaluate(x_test, y_test)
print(f"\nTest Accuracy: {acc:.4f}")
```

B. Moons:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_moons, make_circles, make_blobs
from sklearn.model_selection import train_test_split
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```

```
data, target = make_moons(n_samples=1000, noise=0.1)
```

```
plt.figure(figsize=(5,5))
plt.scatter(data[:,0], data[:,1], c=target, cmap='viridis')
plt.title("Dataset Visualization")
plt.show()
```

```
x_train, x_test, y_train, y_test = train_test_split(data, target, test_size=0.25, random_state=42)
```

```
model = Sequential([
    Dense(8, activation='relu', input_shape=(2,)),
    Dense(4, activation='relu'),
    Dense(1, activation='sigmoid')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.1),
              loss=tf.keras.losses.BinaryCrossentropy(),
              metrics=['accuracy'])
```

```
history = model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
```

```
Epoch 1/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - accuracy: 0.6033 - loss: 0.6593 - val_accuracy: 0.7560 - val_loss: 0.6242
Epoch 2/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7151 - loss: 0.6068 - val_accuracy: 0.7600 - val_loss: 0.5748
Epoch 3/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7383 - loss: 0.5656 - val_accuracy: 0.7840 - val_loss: 0.5235
Epoch 4/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7630 - loss: 0.5272 - val_accuracy: 0.8120 - val_loss: 0.4752
Epoch 5/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7910 - loss: 0.4778 - val_accuracy: 0.8400 - val_loss: 0.4333
```

```
loss, acc = model.evaluate(x_test, y_test)
print(f"\nTest Accuracy: {acc:.4f}")
```

C. Blobs:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_moons, make_circles, make_blobs
from sklearn.model_selection import train_test_split
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```

```
data, target = make_blobs(n_samples=1000, centers=2, random_state=42, cluster_std=2.0)
```

```
plt.figure(figsize=(5,5))
plt.scatter(data[:,0], data[:,1], c=target, cmap='viridis')
plt.title("Dataset Visualization")
plt.show()
```

```
x_train, x_test, y_train, y_test = train_test_split(data, target, test_size=0.25, random_state=42)
```

```
model = Sequential([
    Dense(8, activation='relu', input_shape=(2,)),
    Dense(4, activation='relu'),
    Dense(1, activation='sigmoid')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.1),
              loss=tf.keras.losses.BinaryCrossentropy(),
              metrics=['accuracy'])
```

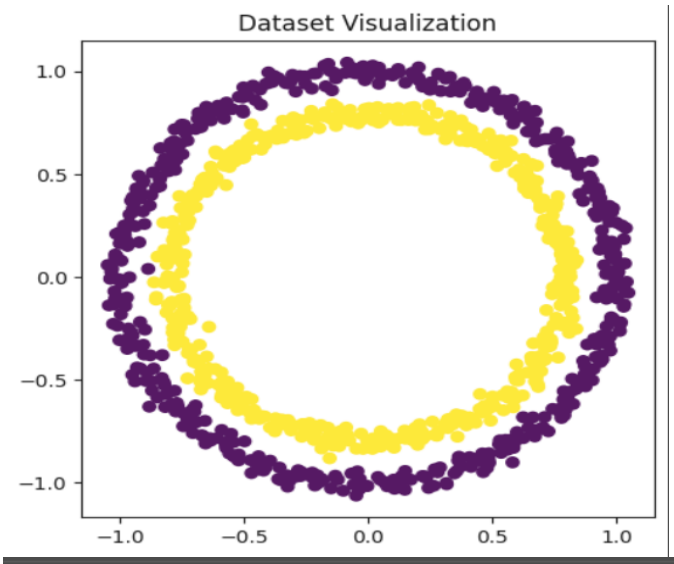
```
history = model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
```

```
Epoch 1/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.6369 - loss: 0.6321 - val_accuracy: 0.9680 - val_loss: 0.3624
Epoch 2/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9598 - loss: 0.3273 - val_accuracy: 0.9920 - val_loss: 0.2346
Epoch 3/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9674 - loss: 0.2314 - val_accuracy: 0.9920 - val_loss: 0.1685
Epoch 4/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9762 - loss: 0.1749 - val_accuracy: 0.9880 - val_loss: 0.1270
Epoch 5/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9722 - loss: 0.1419 - val_accuracy: 0.9880 - val_loss: 0.1003
```

```
loss, acc = model.evaluate(x_test, y_test)
print(f"\nTest Accuracy: {acc:.4f}")
```

4. Result/Output:

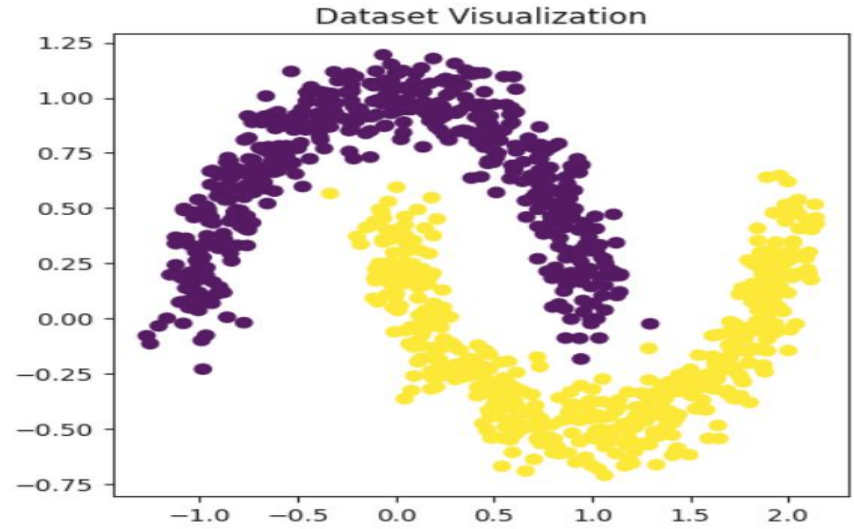
A.Circles:




```
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.4065 - loss: 0.6985

Test Accuracy: 0.4560
```

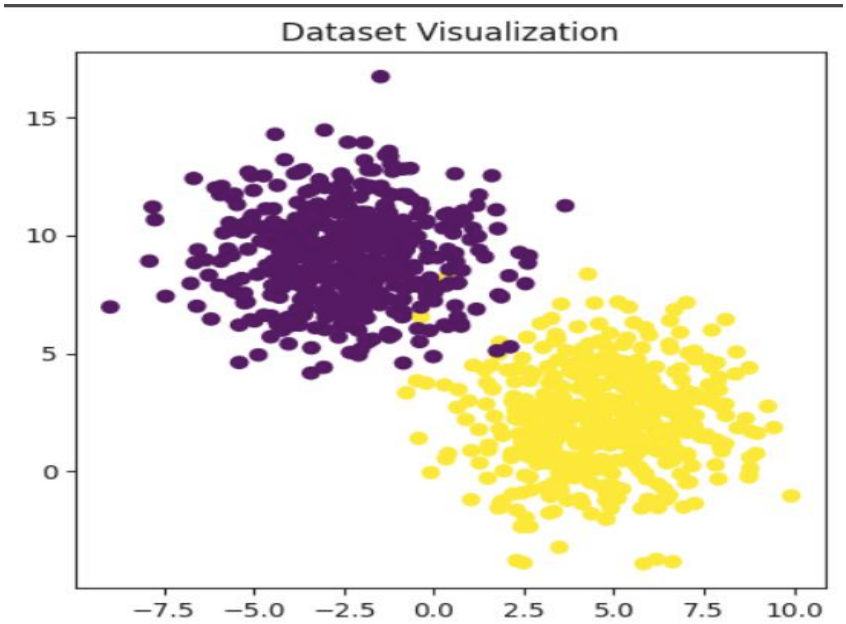
B.Moons:




```
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5300 - loss: 0.6928

Test Accuracy: 0.4880
```

C. Blobs:




```
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 1.0000 - loss: 0.1025

Test Accuracy: 1.0000
```

5. Learning outcomes (What I have learnt):

- Learn to build and train an Artificial Neural Network (ANN) using TensorFlow/Keras.
- Understand how to preprocess data, including normalization of inputs and one-hot encoding of outputs.
- Gain knowledge on how the network learns via forward and backward propagation to make predictions.